

HQL Lab Guide - Sorting, Pagination & Aggregates

Overview

This lab focuses on advanced HQL operations including sorting, pagination, filtering, and aggregate functions using the Product entity from Skill 2.

#SKILL - 3 → Working with HQL - Sorting, Pagination & Aggregates

Prerequisites:

- Basic knowledge of Hibernate
- Basic idea of HQL syntax

In this lab, you will extend the Hibernate module created in **Skill 2** and perform advanced data retrieval using **HQL**, including **sorting**, **pagination**, **filtering**, and **aggregate functions**. These operations help in retrieving structured and meaningful data from the database using Hibernate.

Tasks:

1. Use the **Product entity** created in Skill 2.
2. Add **5–8 additional Product records** into the database.
3. Write HQL queries to retrieve all products sorted by price:
 - a. Ascending order
 - b. Descending order
4. Write an HQL query to sort products by quantity (highest first).
5. Implement pagination using HQL to display:
 - a. First 3 products
 - b. Next 3 products
6. Write HQL queries for aggregate operations:
 - a. Count total number of products
 - b. Count products where quantity > 0
 - c. Count products grouped by description
 - d. Find minimum and maximum price
7. Write an HQL query using **GROUP BY** to group products by description.
8. Write an HQL query using **WHERE** to filter products within a price range.
9. Write HQL queries using **LIKE** such as:
 - a. Names **starting with** certain letters
 - b. Names **ending with** certain letters
 - c. Names containing a pattern **anywhere** (substring)
 - d. Names with an **exact character length**
10. Push the Hibernate project to *GitHub*

Task 1: Use the Product Entity from Skill 2

Solution:

Ensure your **Product.java** entity is properly set up with the following structure:

```
@Entity
@Table(name = "products")
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private Double price;
    private Integer quantity;
    private String description;

    // Constructors
    public Product() {}

    public Product(String name, Double price, Integer quantity, String description) {
        this.name = name;
        this.price = price;
        this.quantity = quantity;
        this.description = description;
    }

    // Getters and Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public Double getPrice() { return price; }
    public void setPrice(Double price) { this.price = price; }

    public Integer getQuantity() { return quantity; }
    public void setQuantity(Integer quantity) { this.quantity = quantity; }

    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }

    @Override
    public String toString() {
        return "Product [id=" + id + ", name=" + name + ", price=" + price +
            ", quantity=" + quantity + ", description=" + description + "]"
    }
}
```

Task 2: Add 5-8 Additional Product Records

Solution:

Create a method to insert sample products:

```
public class ProductDataLoader {

    public static void loadSampleProducts(Session session) {
        Transaction transaction = session.beginTransaction();

        // Sample products
        session.save(new Product("Laptop", 899.99, 15, "Electronics"));
        session.save(new Product("Mouse", 25.50, 50, "Electronics"));
        session.save(new Product("Keyboard", 45.00, 30, "Electronics"));
        session.save(new Product("Monitor", 299.99, 20, "Electronics"));
        session.save(new Product("Desk Chair", 150.00, 0, "Furniture"));
        session.save(new Product("Desk Lamp", 35.75, 25, "Furniture"));
        session.save(new Product("Notebook", 5.99, 100, "Stationery"));
        session.save(new Product("Pen Set", 12.50, 75, "Stationery"));

        transaction.commit();
        System.out.println("Sample products added successfully!");
    }
}
```

Main method to execute:

```
public static void main(String[] args) {
    SessionFactory factory = HibernateUtil.getSessionFactory();
    Session session = factory.openSession();

    ProductDataLoader.loadSampleProducts(session);

    session.close();
    factory.close();
}
```

Task 3: Sort Products by Price

3a. Ascending Order

HQL Query:

```
public static void sortProductsByPriceAscending(Session session) {
    String hql = "FROM Product p ORDER BY p.price ASC";
    Query<Product> query = session.createQuery(hql, Product.class);
}
```

```

        List<Product> products = query.list();

        System.out.println("\n=== Products Sorted by Price (Ascending) ===");
        for (Product product : products) {
            System.out.println(product);
        }
    }
}

```

Expected Output:

```

Notebook - $5.99
Pen Set - $12.50
Mouse - $25.50
Desk Lamp - $35.75
Keyboard - $45.00
Desk Chair - $150.00
Monitor - $299.99
Laptop - $899.99

```

3b. Descending Order

HQL Query:

```

public static void sortProductsByPriceDescending(Session session) {
    String hql = "FROM Product p ORDER BY p.price DESC";
    Query<Product> query = session.createQuery(hql, Product.class);
    List<Product> products = query.list();

    System.out.println("\n=== Products Sorted by Price (Descending) ===");
    for (Product product : products) {
        System.out.println(product);
    }
}

```

Expected Output:

```

Laptop - $899.99
Monitor - $299.99
Desk Chair - $150.00
Keyboard - $45.00
Desk Lamp - $35.75
Mouse - $25.50
Pen Set - $12.50
Notebook - $5.99

```

Task 4: Sort Products by Quantity (Highest First)

HQL Query:

```

public static void sortProductsByQuantityDescending(Session session) {
    String hql = "FROM Product p ORDER BY p.quantity DESC";
    Query<Product> query = session.createQuery(hql, Product.class);
    List<Product> products = query.list();

    System.out.println("\n=== Products Sorted by Quantity (Highest First) ===");
    for (Product product : products) {
        System.out.println(product.getName() + " - Quantity: " + product.getQuantity());
    }
}

```

Expected Output:

```

Notebook - Quantity: 100
Pen Set - Quantity: 75
Mouse - Quantity: 50
Keyboard - Quantity: 30
Desk Lamp - Quantity: 25
Monitor - Quantity: 20
Laptop - Quantity: 15
Desk Chair - Quantity: 0

```

Task 5: Implement Pagination

5a. First 3 Products

HQL Query:

```

public static void getFirstThreeProducts(Session session) {
    String hql = "FROM Product p";
    Query<Product> query = session.createQuery(hql, Product.class);
    query.setFirstResult(0); // Start from index 0
    query.setMaxResults(3);  // Get 3 records

    List<Product> products = query.list();

    System.out.println("\n=== First 3 Products ===");
    for (Product product : products) {
        System.out.println(product);
    }
}

```

5b. Next 3 Products

HQL Query:

```

public static void getNextThreeProducts(Session session) {
    String hql = "FROM Product p";
    Query<Product> query = session.createQuery(hql, Product.class);
    query.setFirstResult(3); // Start from index 3 (skip first 3)
    query.setMaxResults(3); // Get 3 records

    List<Product> products = query.list();

    System.out.println("\n=== Next 3 Products ===");
    for (Product product : products) {
        System.out.println(product);
    }
}

```

Generic Pagination Method:

```

public static void getPaginatedProducts(Session session, int pageNumber, int
pageSize) {
    String hql = "FROM Product p";
    Query<Product> query = session.createQuery(hql, Product.class);
    query.setFirstResult((pageNumber - 1) * pageSize);
    query.setMaxResults(pageSize);

    List<Product> products = query.list();

    System.out.println("\n=== Page " + pageNumber + " (Size: " + pageSize + "
) ===");
    for (Product product : products) {
        System.out.println(product);
    }
}

```

Task 6: Aggregate Operations

6a. Count Total Number of Products

HQL Query:

```

public static void countTotalProducts(Session session) {
    String hql = "SELECT COUNT(p) FROM Product p";
    Query<Long> query = session.createQuery(hql, Long.class);
    Long count = query.uniqueResult();

    System.out.println("\n=== Total Number of Products ===");
    System.out.println("Total Products: " + count);
}

```

Expected Output:

Total Products: 8

6b. Count Products Where Quantity > 0

HQL Query:

```
public static void countProductsInStock(Session session) {  
    String hql = "SELECT COUNT(p) FROM Product p WHERE p.quantity > 0";  
    Query<Long> query = session.createQuery(hql, Long.class);  
    Long count = query.uniqueResult();  
  
    System.out.println("\n=== Products in Stock (Quantity > 0) ===");  
    System.out.println("Products in Stock: " + count);  
}
```

Expected Output:

Products in Stock: 7

6c. Count Products Grouped by Description

HQL Query:

```
public static void countProductsByDescription(Session session) {  
    String hql = "SELECT p.description, COUNT(p) FROM Product p GROUP BY p.description";  
    Query<Object[]> query = session.createQuery(hql, Object[].class);  
    List<Object[]> results = query.list();  
  
    System.out.println("\n=== Products Grouped by Description ===");  
    for (Object[] result : results) {  
        String description = (String) result[0];  
        Long count = (Long) result[1];  
        System.out.println(description + ": " + count + " products");  
    }  
}
```

Expected Output:

Electronics: 4 products
Furniture: 2 products
Stationery: 2 products

6d. Find Minimum and Maximum Price

HQL Query:

```
public static void findMinMaxPrice(Session session) {  
    String hql = "SELECT MIN(p.price), MAX(p.price) FROM Product p";  
    Query<Object[]> query = session.createQuery(hql, Object[].class);  
    Object[] result = query.uniqueResult();
```

```

        Double minPrice = (Double) result[0];
        Double maxPrice = (Double) result[1];

        System.out.println("\n=== Price Range ===");
        System.out.println("Minimum Price: $" + minPrice);
        System.out.println("Maximum Price: $" + maxPrice);
    }
}

```

Expected Output:

```

Minimum Price: $5.99
Maximum Price: $899.99

```

Task 7: GROUP BY to Group Products by Description

HQL Query:

```

public static void groupProductsByDescription(Session session) {
    String hql = "SELECT p.description, p.name, p.price FROM Product p ORDER BY p.description";
    Query<Object[]> query = session.createQuery(hql, Object[].class);
    List<Object[]> results = query.list();

    System.out.println("\n=== Products Grouped by Description ===");
    String currentDescription = "";
    for (Object[] result : results) {
        String description = (String) result[0];
        String name = (String) result[1];
        Double price = (Double) result[2];

        if (!description.equals(currentDescription)) {
            System.out.println("\n" + description + ":");
            currentDescription = description;
        }
        System.out.println("  - " + name + " (" + price + ")");
    }
}

```

Alternative with Aggregation:

```

public static void groupProductsWithAggregation(Session session) {
    String hql = "SELECT p.description, COUNT(p), AVG(p.price), SUM(p.quantity) " +
        "FROM Product p GROUP BY p.description";
    Query<Object[]> query = session.createQuery(hql, Object[].class);
    List<Object[]> results = query.list();

    System.out.println("\n=== Product Statistics by Category ===");
    for (Object[] result : results) {

```



```

        String description = (String) result[0];
        Long count = (Long) result[1];
        Double avgPrice = (Double) result[2];
        Long totalQuantity = (Long) result[3];

        System.out.println(description + ":");
        System.out.println("    Count: " + count);
        System.out.println("    Avg Price: $" + String.format("%.2f", avgPrice)
    );
        System.out.println("    Total Quantity: " + totalQuantity);
    }
}

```

Task 8: Filter Products Within a Price Range

HQL Query:

```

public static void filterProductsByPriceRange(Session session, double minPrice, double maxPrice) {
    String hql = "FROM Product p WHERE p.price BETWEEN :minPrice AND :maxPrice";
    Query<Product> query = session.createQuery(hql, Product.class);
    query.setParameter("minPrice", minPrice);
    query.setParameter("maxPrice", maxPrice);

    List<Product> products = query.list();

    System.out.println("\n=== Products Between $" + minPrice + " and $" + maxPrice + " ===");
    for (Product product : products) {
        System.out.println(product.getName() + " - $" + product.getPrice());
    }
}

```

Usage Example:

```

// Find products between $20 and $100
filterProductsByPriceRange(session, 20.0, 100.0);

```

Expected Output:

```

Products Between $20.0 and $100.0:
Mouse - $25.50
Desk Lamp - $35.75
Keyboard - $45.00

```

Task 9: HQL Queries Using LIKE

9a. Names Starting with Certain Letters

HQL Query:

```
public static void findProductsStartingWith(Session session, String prefix) {
    String hql = "FROM Product p WHERE p.name LIKE :pattern";
    Query<Product> query = session.createQuery(hql, Product.class);
    query.setParameter("pattern", prefix + "%");

    List<Product> products = query.list();

    System.out.println("\n=== Products Starting with '" + prefix + "' ===");
    for (Product product : products) {
        System.out.println(product.getName());
    }
}
```

Usage Examples:

```
findProductsStartingWith(session, "D"); // Desk Chair, Desk Lamp
findProductsStartingWith(session, "M"); // Mouse, Monitor
```

9b. Names Ending with Certain Letters

HQL Query:

```
public static void findProductsEndingWith(Session session, String suffix) {
    String hql = "FROM Product p WHERE p.name LIKE :pattern";
    Query<Product> query = session.createQuery(hql, Product.class);
    query.setParameter("pattern", "%" + suffix);

    List<Product> products = query.list();

    System.out.println("\n=== Products Ending with '" + suffix + "' ===");
    for (Product product : products) {
        System.out.println(product.getName());
    }
}
```

Usage Examples:

```
findProductsEndingWith(session, "p"); // Laptop, Desk Lamp
findProductsEndingWith(session, "set"); // Pen Set
```

9c. Names Containing a Pattern Anywhere (Substring)

HQL Query:

```
public static void findProductsContaining(Session session, String substring)
{
```

```

String hql = "FROM Product p WHERE p.name LIKE :pattern";
Query<Product> query = session.createQuery(hql, Product.class);
query.setParameter("pattern", "%" + substring + "%");

List<Product> products = query.list();

System.out.println("\n=== Products Containing '" + substring + "' ===");
for (Product product : products) {
    System.out.println(product.getName());
}
}

```

Usage Examples:

```

findProductsContaining(session, "Desk"); // Desk Chair, Desk Lamp
findProductsContaining(session, "oo");   // Notebook

```

9d. Names with an Exact Character Length

HQL Query:

```

public static void findProductsByNameLength(Session session, int length) {
    String hql = "FROM Product p WHERE LENGTH(p.name) = :length";
    Query<Product> query = session.createQuery(hql, Product.class);
    query.setParameter("length", length);

    List<Product> products = query.list();

    System.out.println("\n=== Products with Name Length " + length + " ===");
    for (Product product : products) {
        System.out.println(product.getName() + " (Length: " + product.getName()
        ().length() + ")");
    }
}

```

Alternative using LIKE with underscores:

```

public static void findProductsByExactLengthPattern(Session session, int length) {
    StringBuilder pattern = new StringBuilder();
    for (int i = 0; i < length; i++) {
        pattern.append("_");
    }

    String hql = "FROM Product p WHERE p.name LIKE :pattern";
    Query<Product> query = session.createQuery(hql, Product.class);
    query.setParameter("pattern", pattern.toString());

    List<Product> products = query.list();

    System.out.println("\n=== Products with Exactly " + length + " Characters

```

```
===");  
    for (Product product : products) {  
        System.out.println(product.getName());  
    }  
}
```

Usage Example:

```
findProductsByNameLength(session, 5);    // Mouse  
findProductsByNameLength(session, 7);    // Monitor, Pen Set
```

Task 10: Push Hibernate Project to GitHub

Step-by-Step Guide:

Step 1: Create .gitignore File

Create a .gitignore file in your project root:

```
# Maven  
target/  
pom.xml.tag  
pom.xml.releaseBackup  
pom.xml.versionsBackup  
pom.xml.next  
release.properties  
  
# Eclipse  
.classpath  
.project  
.settings/  
  
# IntelliJ IDEA  
.idea/  
*.iml  
*.iws  
*.ipr  
out/  
  
# NetBeans  
nbproject/private/  
build/  
nbbuild/  
dist/  
nbdist/  
  
# Logs  
*.log
```

```
# Database
*.db
*.sqlite
```

```
# OS
.DS_Store
Thumbs.db
```

Step 2: Initialize Git Repository

```
cd your-project-directory
git init
```

Step 3: Add Files to Git

```
git add .
git status
```

Step 4: Create Initial Commit

```
git commit -m "Initial commit: HQL Skill 3 - Sorting, Pagination & Aggregates"
```

Step 5: Create Repository on GitHub

1. Go to GitHub.com
2. Click "New Repository"
3. Name it: hibernate-hql-skill3
4. Don't initialize with README (since you already have code)
5. Click "Create Repository"

Step 6: Connect Local to Remote

```
git remote add origin https://github.com/your-username/hibernate-hql-skill3.git
git branch -M main
git push -u origin main
```

Step 7: Create README.md

Create a README.md file:

```
# Hibernate HQL Skill 3 - Sorting, Pagination & Aggregates
```

Overview

This project demonstrates advanced HQL operations including:

- Sorting (ASC/DESC)
- Pagination
- Aggregate functions (COUNT, MIN, MAX, AVG, SUM)
- GROUP BY operations
- Filtering with WHERE
- Pattern matching with LIKE

Technologies

- Java 8+
- Hibernate 5.x
- MySQL/PostgreSQL
- Maven

Setup

1. Clone the repository
2. Update database credentials in `hibernate.cfg.xml`
3. Run `ProductDataLoader` to populate sample data
4. Execute queries from `HQLDemo` class

Tasks Completed

- ✓ Product entity setup
- ✓ Sample data insertion
- ✓ Sorting by price and quantity
- ✓ Pagination implementation
- ✓ Aggregate operations
- ✓ GROUP BY queries
- ✓ Price range filtering
- ✓ LIKE pattern matching

Author

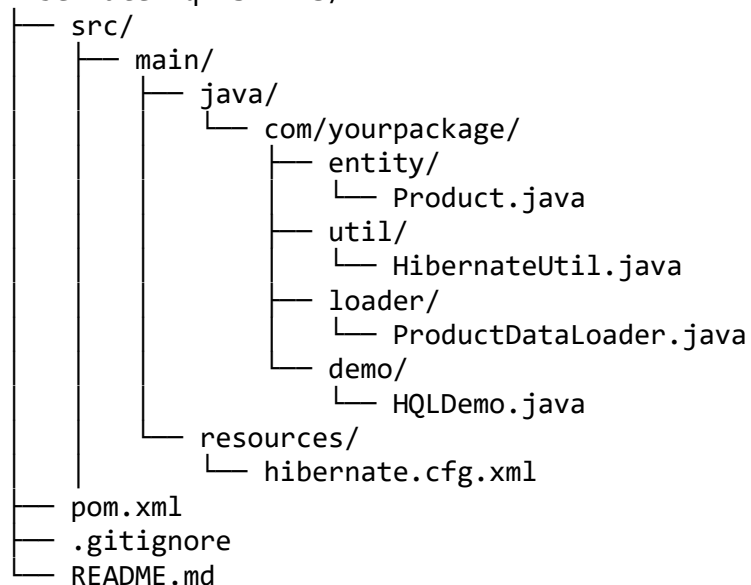
[Your Name]

Step 8: Push README

```
git add README.md
git commit -m "Add README documentation"
git push
```

Project Structure on GitHub:

hibernate-hql-skill3/



Complete Demo Class

Here's a complete demo class that runs all queries:

```
public class HQLDemo {

    public static void main(String[] args) {
        SessionFactory factory = HibernateUtil.getSessionFactory();
        Session session = factory.openSession();

        try {
            // Task 2: Load sample data (run once)
            // ProductDataLoader.loadSampleProducts(session);

            // Task 3a: Sort by price ascending
            sortProductsByPriceAscending(session);

            // Task 3b: Sort by price descending
            sortProductsByPriceDescending(session);

            // Task 4: Sort by quantity
            sortProductsByQuantityDescending(session);

            // Task 5a: First 3 products
            getFirstThreeProducts(session);

            // Task 5b: Next 3 products
            getNextThreeProducts(session);

            // Task 6a: Count total products
            countTotalProducts(session);

            // Task 6b: Count products in stock
            countProductsInStock(session);

            // Task 6c: Count by description
            countProductsByDescription(session);

            // Task 6d: Min/Max price
            findMinMaxPrice(session);

            // Task 7: Group by description
            groupProductsByDescription(session);

            // Task 8: Price range filter
            filterProductsByPriceRange(session, 20.0, 100.0);
        }
    }
}
```

```
        // Task 9a: Names starting with 'D'
        findProductsStartingWith(session, "D");

        // Task 9b: Names ending with 'p'
        findProductsEndingWith(session, "p");

        // Task 9c: Names containing 'Desk'
        findProductsContaining(session, "Desk");

        // Task 9d: Names with exact Length
        findProductsByNameLength(session, 5);

    } finally {
        session.close();
        factory.close();
    }
}

// ... (all the methods defined above)
}
```

Testing Checklist

- ☐ Product entity properly configured
 - ☐ 8 sample products inserted
 - ☐ Sorting by price (ASC/DESC) works
 - ☐ Sorting by quantity works
 - ☐ Pagination returns correct results
 - ☐ All aggregate functions execute successfully
 - ☐ GROUP BY groups products correctly
 - ☐ Price range filtering works
 - ☐ All LIKE patterns match correctly
 - ☐ Project pushed to GitHub with proper .gitignore
 - ☐ README.md created and informative
-

Additional Resources

- [Hibernate HQL Documentation](#)
 - [HQL vs SQL Differences](#)
 - [Hibernate Query Language Guide](#)
-

End of Lab Guide